Testing a buy at its boundaries

Studied 18 October 2026; integrated 19 October. Task: L001e, boundary verification. Sidiq review status: Not yet reviewed.

Research question and hypothesis

Does the unchanged C++ buy function agree with an independent calculation at exact cash and position limits, just inside them and just outside them? Before running the checks, the prediction was: exact affordability succeeds, one unit less fails, and every rejected request preserves both cash and shares. The same equality rule applies when a buy reaches the 10000-share policy cap.

Yesterday introduced try_buy; today studies one concept: boundary testing. There is no new trading operation or C++ language lesson. The Python verification machinery generates synthetic inputs and compiles a temporary C++ driver around the existing function. There is no market history, training period, parameter optimisation or profitability measurement. We test an accounting contract.

Beginner explanation and vocabulary

A boundary is where the permitted outcome changes. Testing an ordinary buy with plenty of cash can miss a bug that rejects a perfectly affordable buy at exactly zero remaining cash. A useful pattern is one below, exactly at, and one above the threshold. We inspect the return flag and both final fields; a rejection flag alone would miss a hidden cash deduction.

An oracle is an independently calculated expected answer. It is fallible, so six hand-calculated anchors check it before generated cases run. Independence here means different arithmetic and control structure, not independent people or a mathematically verified implementation. Both implementations still use the same written policy, which could itself contain a mistake.

A Cartesian product selects every combination from several lists. With six values for each of five inputs, there are 6*6*6*6*6 = 7776 tuples. Deduplication removes repeated tuples from the added threshold examples. Deterministic sorting makes failures and input hashes reproducible. The generator is verification tooling; learning its loop syntax is not an additional lesson today.

Contract, units and hand example

Cash C, starting shares q and fee f are whole integers from 0 to 10000. Quantity n and price p are whole integers from 1 to 10000. The final share count cannot exceed 10000. There is no debt, short selling or fractional-money rounding. The cap is our teaching policy, not a market rule. The source accepts typed integers; malformed text and implicit fractional conversion are separate.

D = n*p + f; candidate_cash = C-D; candidate_shares = q+n. An order is accepted only when inputs satisfy the policy, candidate_cash >= 0 and candidate_shares <= 10000. Rejection returns the original C and q without charging a fee.

Starting cash	Buy and fee	Expected outcome
102	2 shares at 50, fee 3	Reject; cash 102 and shares 4 unchanged
103	2 shares at 50, fee 3	Accept; cash 0 and shares 6
104	2 shares at 50, fee 3	Accept; cash 1 and shares 6

Why the reference is different

The C++ function reserves the fee locally, divides remaining cash by positive price and checks quantity against that quotient before multiplication. It checks remaining position capacity by subtracting starting shares from the cap. These guards keep fixed-width arithmetic safe.

The Python reference directly computes candidate_cash = C - n*p - f and candidate_shares = q+n, then tests their final limits. Python's integers support this small direct calculation without fixed-width overflow. It does not copy the C++ division or remaining-capacity branches. For every accepted reference result, it also verifies C_new + q_new*p = C + q*p - f.

The identity uses the same fill price as the unchanged mark. Buying transfers value from cash to holdings; only the fee reduces marked wealth under that assumption. If the mark differs from the fill, the identity changes. A correct integer ledger cannot establish a realistic executable price.

Selected cases and evaluation protocol

The edge grid uses -1, 0, 1, 9999, 10000 and 10001 for each of cash, shares, quantity, price and fee. This exercises every input's policy edges, including negative price omitted from yesterday's 14 fixed examples. It includes invalid starting state, which must be rejected without repair.

Additional affordability examples select n from 1,2,7,99; p from 1,3,50,101; f from 0,1,3; and q from 0,4,9999,10000. For each combination, C takes D-1, D and D+1. A tuple can still reject at D because another rule fails, such as the position cap. The oracle evaluates the entire contract. Further cases place q one below, at and above 10000-n for quantities 1,2,7,9999,10000 at price 1, fee 0 and cash 10000. Together these add 575 unique tuples to the grid: 8351 total.

Each tuple starts with a fresh AccountState. Tests do not carry state from one order to the next. A temporary C++17 driver feeds those known integer tuples into the unchanged production function and prints the flag, cash and shares. It is a test harness, not a production text parser. The verifier checks output count, every output line, empty stderr and exit zero, twice per build.

Two separate verifier runs freshly compile with warnings as errors and UndefinedBehaviorSanitizer, then reproduce identical JSON. Input, expected-output and production-source SHA-256 hashes are saved. The correct function is never overwritten by a mutation. No random seed or market download is used.

What passing and failing checks teach

All 8351 comparisons pass: 250 accepted and 8101 rejected with unchanged state. These are counts of selected tests, not market probabilities, statistical confidence or measured code coverage. The two builds each execute the complete input set twice without sanitizer diagnostics.

Two wrong copies are rejected: changing the cash guard from > to >= and making the position guard exclusive. Both compile but fail on an intended boundary mismatch. A third mutation, changing the positive-price guard to reject only zero, unexpectedly passes. For negative price, nonnegative available cash divided by price is nonpositive, so positive quantity still rejects at the later affordability guard. It is redundant rejection for these inputs, not evidence that negative prices were accepted. The initial control expectation was wrong and is recorded. A real Python -O invocation is refused so disabled assertions cannot claim success.

Reproduce, costs and limitations

From the repository root run: `.venv/bin/python scripts/verify_buy_boundaries.py`. The source defaults to cpp/projects/portfolio_accounting/validated_buy.cpp. The driver and binary live in an automatically removed temporary directory. A compiler supporting the configured sanitizer is required. Integration adds make cpp-buy-boundaries to make check; remote CI status is recorded separately.

Full make check passes 608 Python tests, Ruff, formatting, strict package typing and all existing C++ checks, including the newly integrated validated-buy target. The boundary verifier also passes separate Ruff lint/format checks. One initial lint failure was an overlong description string; it was shortened before the first executable run. No production failure or dependency change occurred. The existing pip cache warning is nonfatal; dependency consistency passes.

A Cartesian grid is exhaustive only for its selected six-value lists. The full contract admits many other integer values, and the tests omit sequences, sale orders, partial fills, concurrency, execution delay and future-price invariance. The arithmetic reasoning in yesterday's lesson still matters; a sanitizer only examines executed paths. We make no machine-maximum-input proof here.

The fee remains a specified whole-unit amount, charged only for acceptance. Spread, slippage, impact, financing, taxes and settlement are absent. Unknown fills and real information timing need separate models. More passing synthetic cases provide no evidence of an investment edge. The chosen 10000 cap restricts the experiment and must not silently become a broker risk limit.

Interview explanation

I test both sides of affordability and position thresholds, plus equality. The C++ implementation uses safe division and capacity guards; my Python oracle evaluates the final ledger directly. I compare acceptance and both final fields, including unchanged rejected state. Hand anchors and two detected mutations and one surviving control examine the verifier. The result is bounded software evidence, not proof of every input, complete transaction semantics, information timing or profitable trading.

Progress and next task

On October 18 the one-commit allowance integrated October 17's buy lesson, so this boundary lesson was saved locally. October 19 integrates it after all ten original inputs and seven artifacts matched their hashes and sizes. Current fresh builds reproduce the original results; the two detected mutations and the survivor are rechecked. Original study evidence is retained. Today's one-commit allowance covers this integration; the weekly synthesis remains pending.

Next learning is L001f: reproduce and consolidate the accounting examples without adding a new trade feature. L002 decision/execution ordering and future-price invariance remain separate and unfinished. October 19 trend implementation remains conditional on those and applicable return prerequisites. C003b/C004/C005 and Q004a/Q005b real-data gates are preserved. No user input blocks synthetic continuation. Do not repeat L001e merely because the tracked plan still says planned.

Continue the existing 08:00 Europe/London schedule, including daylight saving, through December 31. After that date stop new research and request the next programme scope. Sidiq's personal review remains Not yet reviewed; successful tests do not establish personal mastery.

Primary sources consulted on October 18 and 19

Python, itertools.product: documents Cartesian combinations. The selected edge lists and threshold protocol are project choices, not prescribed by this reference. https://docs.python.org/3/library/itertools.html#itertools.product

Microsoft, Multiplicative Operators: explains integer division and its operand constraints. Our alternative final-ledger oracle is a project derivation. https://learn.microsoft.com/en-us/cpp/cpp/multiplicative-operators-and-the-modulus-operator?view=msvc-170

Clang, UndefinedBehaviorSanitizer: documents compile/link instrumentation and runtime checks. The absence of diagnostics here applies to the executed synthetic cases only. https://clang.llvm.org/docs/UndefinedBehaviorSanitizer.html

Python, Numeric Types: documents unlimited-precision integers, used by the direct-product oracle. https://docs.python.org/3/library/stdtypes.html#numeric-types-int-float-complex

Check your understanding

1. Why test cash 102, 103 and 104 for the same two-share order with fee 3?

2. Cash is 1, shares are 9999, quantity is 1, price is 1 and fee is 0. What is the result? What changes if starting shares are 10000?

3. Why is the Python oracle less likely to share an equality bug with the C++ guard?

4. Does a rejection flag by itself prove that the account did not change?

5. Do 8351 passing cases or a quiet sanitizer establish an exhaustive proof or profitable strategy?

Suggested answers

1. The debit is 103. These inputs isolate insufficient cash, exact affordability and one unit remaining. An incorrect exclusive comparison can pass ordinary buys but reject the equality case.

2. The first buy succeeds with cash 0 and shares 10000. With 10000 starting shares it exceeds the position cap and rejects, preserving cash 1 and shares 10000. Affordability alone is insufficient.

3. It calculates the candidate final cash/share ledger using Python integers instead of copying the quotient and remaining-capacity guards. Hand anchors add another check. The shared policy can still be wrong, and both implementations remain fallible.

4. No. A function could deduct a fee and then return false. Check both original fields as well as the return flag; a rejection is supposed to preserve the entire two-field state here.

5. Neither. The grid covers selected values, while the sanitizer examines executed paths. Neither models market fills, timing or returns. Preserve mathematical reasoning and separate empirical gates.